Terminal-Bench on AgentCore

One task, one image, one Runtime; a fresh microVM per episode

Kaige Yang
Senior Applied Scientist, AWS

September 29, 2026

Agenda

00 Agentic RL on AgentCore

Architecture, slime hooks, AgentCore concepts in RL terms

01 Task and dataset

What a task is, 166 rows, 218 images, sizes, why one image per row

02 Building the images

How many, what we add, step by step

03 Running on AgentCore microVMs

Session model, one episode, step by step

04 Cold start and pre-open

Cold start per image, why and how we pre-open, with vs without

05 Heartbeat

Why sessions need it, how it works, what it saved

06 Smoke run results

Health, sandbox cost, learning signal, next steps

07 SWE-bench: the same analysis

Rows, images, sizes, what runs on AgentCore and why

© 2026, Amazon Web Services, Inc. or its affiliates.

One image per row works on AgentCore; pre-open and heartbeat hide its costs

Table 1 **The deck in one table**

Goal	RL-train Qwen3-4B (slime GRPO) on Terminal-Bench, every episode in its own AgentCore microVM
Data	229 single-container tasks → **218 images and Runtimes** → **166 rows** (135 train, 31 eval), one row per task (slide 8)
Why one image per row	Each task **is** a machine (base OS, packages, broken state); 48 GB in total, p50 127 MB, max 2.0 GB (slides 9–10)
Build	Task Dockerfile + one wrapper layer, arm64, $\leq$ 2 GB, one ECR repo + one Runtime each; 39.6 h of builds, 16 in parallel (slide 11)
Challenge	A fresh microVM per episode: only $\approx$ 1 warm VM per Runtime, so 7 of 8 GRPO siblings boot cold, **6 s p50 to 49 s** for 1–2 GB images (slide 18)
Pre-open	Boot the next 2 batches' sessions before slime asks: wait p50 774 → **5 ms**, smoke run `open_wait_ms` **p95 = 0** (slide 22)
Heartbeat	Pre-opened sessions idle up to 16.5 min, past the 15-min reclaim: 17 of 382 would have been lost; with pings, **0 lost** (slide 25)
Smoke run	Runs end to end with 0 infra errors; learning not yet: 4 of 382 solved, reward too sparse for GRPO → partial credit next (slide 26)
SWE-bench	Same pattern, 8$\times$ larger images: 420 of 500 Verified rows fit a microVM today, 27 never can (slide 34)

Source: `code/demos/demo_cpu_tasks/terminalbench/` (`tasks/images.csv`, smoke run 2026-09-23, warm-start probe 2026-09-25); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Terminal-Bench is the case a shared sandbox image can’t cover

Table 2 What this demo covers

Use case covered	Tasks that need **their own OS image** (packages, `/etc`, services): hundreds of custom images, one per task, routed per episode
Why this demo	Most literature RLVR tasks run in one generic image; this is the case they can’t: custom images **at scale**, where a cold start is paid per episode
Isolation	One episode = one **brand-new session** = one dedicated microVM, stopped (VM terminated, memory sanitized) when the episode ends. Nothing leaks between episodes or GRPO siblings
Design considerations	Strict image routing (unmapped task = error, no fallback) · look-ahead pre-open hides the boot · always-on heartbeat keeps pre-opened sessions past the 15-min idle timeout · retry on a new id; infra failures never scored as wrong answers

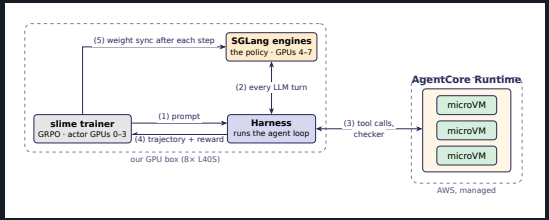
In plain English: every exercise comes with its own computer; each attempt gets a brand-new copy of it, booted before the trainer asks and kept awake until it’s used.

Source: `code/demos/demo_cpu_tasks/terminalbench/`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

The sandbox is the reward source; slime reaches AgentCore through two existing hooks

Figure 1 Agentic RL: trainer, inference, harness, sandbox



- **Trainer** (slime: Megatron, GRPO) updates the policy; **inference** (SGLang) serves it.
- **Harness** runs the agent loop and records the exact token ids + logprobs, so training stays on-policy.
- **Sandbox** (AgentCore) runs the tool calls and the hidden tests: **it produces the reward**.

Table 3 How slime plugs in (no fork)

slime flag	Our code
`--custom-generate-function-path`	`generate.generate`: one episode in a fresh session → one loss-masked Sample
`--custom-rm-path`	reward = hidden tests' result from the sandbox
`--data-source-path`	`LookaheadDataSource`: tells the pool what comes next (slide 20)

- Every sandbox op is one `InvokeAgentRuntime(ARN, session id)`; the agent loop runs **in** the session and dials the trainer's adapter back over the VPC.
- Rollout is **fully async**: episodes keep running while slime trains and syncs weights.

Source: `code/agentcore_slime_demo/generate.py`, `harness_pool.py`; `demo_cpu_tasks/run.sh`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore never sees prompts or rewards: an RL batch is just Runtimes and session ids

Table 4 AgentCore concepts seen from agentic RL

AgentCore	What it is	RL meaning	Terminal-Bench, one step
Image	container in ECR, arm64, $\leq$ 2 GB	the environment	one per task, 218 built
Runtime	one image behind one ARN, deployed once	a task's env, reusable	4 tasks drawn $\rightarrow$ 4 Runtimes
Session	one `runtimeSessionId`; the first invoke boots it	one rollout (episode)	4 $\times$ 8 = **32 new sessions**
microVM	Firecracker VM per session, 2 vCPU / 8 GB	isolation of that rollout	32 VMs, stopped at episode end
GRPO group	8 sessions on **one** Runtime	same start state, no sharing	8 separate VMs per task

Table 5 Lifecycle facts that shape the RL design

No "create session" call	a new id's first invoke is a **cold start** $\rightarrow$ pre-open (slide 18)
15-min idle reclaim, silent	a waiting session comes back empty $\rightarrow$ heartbeat (slide 23)
5000 active sessions · 25 new/s · 1000 Runtimes	bounds rollouts in flight, burst rate and distinct images [Q1][Q2]

Source: AgentCore docs [D3] and quotas [Q1][Q2] (slide 36); `demo_cpu_tasks/run.sh` (RB = 4, NS = 8); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

A Terminal-Bench task is a whole machine: instruction, Dockerfile, reference solution and hidden tests

Table 6 **The files of one task (`acl-permissions-inheritance`)**

File	Role
`task.yaml`	Instruction (the prompt), difficulty, category, test timeout
`Dockerfile`	**The environment**: base image, packages, users, files
`solution.sh`	Reference solution (the oracle), never shown to the agent
`tests/`, `run-tests.sh`	Hidden pytest checks, injected only at grading
`docker-compose.yaml`	How Terminal-Bench runs the container

Example. *"Create `/srv/shared` owned by group `research`, setgid, mode 2770, with default ACLs so files made by `alice` or `bob` stay shared."* The Dockerfile installs `acl` and creates `alice`, `bob` and `research`; the tests create files as each user and check access.

Source: `laude-institute/terminal-bench` (main), fetched to `terminalbench/tasks/`; checked 2026-09-29.

- **The agent works on the OS itself:** it installs packages, edits `/etc`, starts services, builds code.
- **Reward = 1 only if every hidden test passes** (`run-tests.sh` exit 0). No LLM judge.
- **The tests read machine state**, so the starting machine must be exactly the one the task author built.

In plain English: the task is not a question, it's a computer in a known state plus a list of checks.

© 2026, Amazon Web Services, Inc. or its affiliates.

241 benchmark tasks become 166 training rows, one row and one image per task

Figure 2 From benchmark to dataset (tasks kept at each stage)

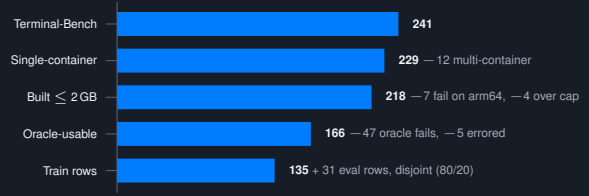


Table 7 What the 229 tasks cover

Difficulty	easy 59 · medium 113 · hard 57
Top categories	software-eng. 46 · sysadmin 25 · data science 19 · sci. computing 18 · security 18 · debugging 17 · +20 more

- **One row = one task.** Its `metadata.task.image` is the task name, which the pool map turns into that task's Runtime ARN.
- **Oracle check**, per image on AgentCore: the reference solution must pass and the untouched machine must fail. Otherwise the reward is structurally 0 or 1.
- **Counts:** 166 rows → 166 images and Runtimes in use; each step draws RB = 4 rows × NS = 8 samples.

Source: `terminalbench/tasks/manifest.json`, `images.csv`, `oracle.csv`; `data/terminalbench_{train,eval}.jsonl`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Half the images are under 127 MB, but a heavy tail reaches the 2 GB cap

Figure 3 Compressed image size of the 218 built Terminal-Bench images (100 MB bins)



- **min 61 MB · p50 127 MB · p90 447 MB · max 2,003 MB · total 48 GB.** 201 of 218 are under 500 MB; 7 are over 1 GB (ML models, QEMU disks, datasets).
- Image size sets the cold boot (slide 18); the 4 dropped `mteb-*` / `png-generation` images are 3× over the cap.

Source: `terminalbench/tasks/images.csv` (`size_mb` = ECR compressed size incl. our 54 MB wrapper, 218 registered); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Each row needs its own image because the environment is part of the task

Table 8 What the 226 task Dockerfiles do on top of their base

Dockerfile step	Tasks
Install packages (`apt`, `pip`, `npm`, ...)	179
Copy task files (source, data, broken configs)	142
Download at build time (`git clone`, `wget`, models)	68
Nothing beyond the base image	14

8 distinct bases: `t-bench/ubuntu-24-04` 104, `t-bench/python-3-13` 92, `python` 20, ...

Table 9 Three examples

`broken-python`	Deletes `pip`, `setuptools`, `wheel`: the broken state **is** the task
`3d-model-format-legacy`	cmake/clang toolchain + a 2007 win32 library to port
`llm-spec-decoding`	torch + two HF models baked in: 2.0 GB

Why not one shared image?

- **The states contradict:** one task needs `pip` gone, another needs it; bases range from Ubuntu to Debian, Alpine and `localstack`.
- **Size:** 48 GB of environments vs a 2 GB cap per image.
- **Reward validity:** tests check machine state, and the oracle check only holds for the exact image it ran on.

Why not install at episode start?

- Builds take minutes to hours (p50 6.8 min) and hit the network; siblings could diverge.

Dedup: 15 base-only tasks share 4 images, so 218 images have 203 distinct digests.

Source: Task Dockerfiles in `terminalbench/tasks/*/Dockerfile` (3 tasks build from `client/`); image digests in `images.csv`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

We build 229 images and ship 218: one ECR repo and one Runtime per task

Table 10 **Build outcome (arm64, 2026-09-23)**

Outcome	Tasks	Detail
Attempted	229	one per single-container task
Built, pushed, registered	**218**	203 distinct digests, 48 GB in ECR
Build failed on arm64	7	x86-only (asm, QEMU x86, amd64 base) 4 · dead download / apt 3
Over 2 GB	4	`mteb-*` ×3, `png-generation`: 6.5–6.7 GB

Table 11 **Build time per image (193 timed builds)**

p50 / p90 / max	6.8 min / 28 min / 2.0 h
Total	**39.6 h** serial, 16 builds in parallel (`TB_BUILD_CONC`)

- **Names:** ECR `acs-tb-<task>`, Runtime `acs_tb_<task>`, map in `tasks/arn_by_image.json`.
- **arm64 because microVMs are arm64-only**; this x86 host cross-builds under QEMU (buildx), which is why builds are slow.
- **218 Runtimes fit the quota** of 1000 per account.
- **Resumable:** an image already pushed with the same wrapper hash is skipped; an existing Runtime is reused.

Source: `terminalbench/tasks/images.csv` (`status`, `build_s`, `digest`); `deploy_terminalbench.py`; Runtime quota [Q1] (slide 36); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

We leave the task's environment untouched and append one 54 MB server layer

`Dockerfile.acs` (task Dockerfile + this, generated per task)

```
# ... the task's own Dockerfile, unchanged ...
FROM ghcr.io/laude-institute/t-bench/ubuntu-24-04:latest
RUN apt-get update && apt-get install -y acl util-linux
RUN groupadd -f research && useradd -m alice ...

# ===== sandbox wrapper (appended by deploy_terminalbench.py)
USER root
COPY --from=acs py /opt/acs/py # standalone Python
COPY --from=acs acs_http_server.py sandbox_server_agent.py /opt/acs/
RUN /opt/acs/py/bin/python3 -c "import http.server, tarfile, json"
EXPOSE 8080
ENTRYPOINT []
CMD ["/opt/acs/py/bin/python3", "/opt/acs/acs_http_server.py"]
```

- **Own Python, not the task's.** Task images range from Python 3.8 to no Python at all, and changing the task's Python would change what gets graded.
- **A stdlib HTTP server** on port 8080 speaks AgentCore's invoke protocol: `exec`, file put/get, boot timestamps.
- `WORKDIR` **kept** (`/app`, `/app/astropy`, ...); the server reports it on the first ping.
- **No tests in the image.** `tests/` and `run-tests.sh` are injected from the trainer after the agent stops.

Source: `terminalbench/deploy_terminalbench.py`, `terminalbench/acs_ctx/`; `tasks/acl-permissions-inheritance/Dockerfile.acs`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Building the pool takes six commands; validate on two tasks first

From `code/demos/demo_cpu_tasks/`, on the host

```
# 0. once: arm64 emulation + a buildx builder (2_build_image.sh does it)
docker run --privileged --rm tonistiigi/binfmt --install arm64

# 1. fetch the benchmark -> terminalbench/tasks/ + manifest.json
python3 terminalbench/fetch_terminalbench.py

# 2. dry run on 2 tasks (real AWS $): build + push + register
VARIANT=terminalbench TB_LIMIT=2 bash 2_build_image.sh
VARIANT=terminalbench TB_LIMIT=2 bash 4_deploy_runtime.sh

# 3. every task: build + push to ECR acs-tb-<task> (16 parallel)
VARIANT=terminalbench bash 2_build_image.sh
VARIANT=terminalbench bash 3_push_ecr.sh # prints images.csv

# 4. one Runtime per image -> tasks/arn_by_image.json
VARIANT=terminalbench bash 4_deploy_runtime.sh
```

0. Needed only on an x86 host.
1. Clones the repo, keeps the 229 single-container tasks.
2. Catches wrapper and IAM problems in minutes, not hours.
3. Writes `status` / `size_mb` / `build_s` per task to `images.csv`; over-cap and failed builds are skipped, not fatal.
4. VPC Runtimes with NAT egress (the tests install `uv` and pytest at grade time) and the SG→SG rule to the adapter ports.

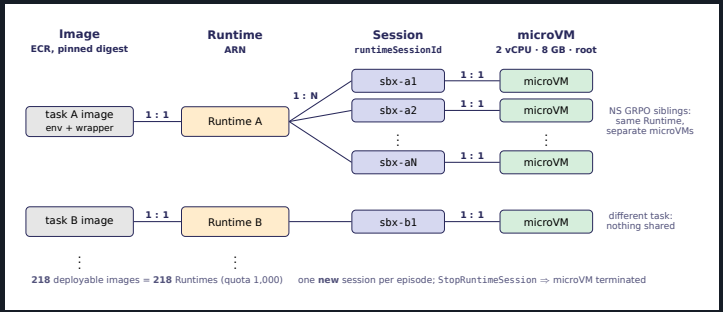
Knobs: `TB_TASKS=a,b`, `TB_BUILD_CONC`, `TB_REBUILD=1`, `SKIP_BUILD=1`, `CONTINUE_ON_ERROR=1`.

Source: `demo_cpu_tasks/README.md` section 3, `deploy/demo.env` (`VARIANT=terminalbench`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

218 tasks, 218 images, 218 Runtimes; every episode gets a new microVM

Figure 4 image 1:1 Runtime 1:N session 1:1 microVM



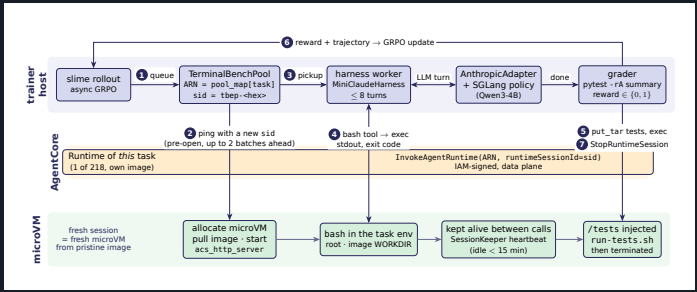
- **Runtime = one image.** Routing is strict: `task.image` → `arn_by_image.json`; an unmapped task stops the run before any GPU work.
- **Session = microVM, 1:1.** A new `runtimeSessionId` boots a fresh Firecracker VM (2 vCPU, 8 GB) from the pristine image; `StopRuntimeSession` terminates and sanitizes it.
- **Sessions per Runtime are never configured.** A task drawn in a step gets NS = 8 sessions; the others get 0.

Source: `demo_cpu_tasks/image_routing.py`, `terminalbench_pool.py`; session model per AgentCore docs [D3] (slide 36); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

One episode: route, fresh session, agent loop in the microVM, grade, stop

Figure 5 route → fresh session → agent loop → grade → stop



1. **Open:** take a pre-opened session for the task, or open one. There is no “create” call: the first invoke with a new id **is** the boot.
2. **Agent loop in the session:** `PROBLEM.md` goes into the task’s workdir; the mini_claude loop runs **inside** the microVM and calls the policy through the trainer’s adapter (VPC dial-back), up to 8 turns.
3. **Grade:** inject `tests/`, run `run-tests.sh`; reward 1 iff every test passes. Then `StopRuntimeSession`.

Source: `agentcore_slime_demo/terminalbench_pool.py` (`_run_episode`, `grade_episode`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Once the pool is deployed, a run is oracle check, data, train, clean up

From `code/demos/demo_cpu_tasks/`, on the host

```
# 5. oracle check on AgentCore -> tasks/oracle.csv
docker exec -e AGENTCORE_REGION=us-west-2 agentcore_p4 python3 \
  /work/code/demos/demo_cpu_tasks/terminalbench/validate_oracle.py

# 6. rows for the usable tasks only -> data/terminalbench_*.jsonl
DATASET=terminalbench bash 1_prepare_data.sh

# 7. train: 10 steps + 1 eval (slime inside agentcore_p4)
DATASET=terminalbench bash 5_run.sh
# -> logs/<tag>.{log,csv,eval.csv,png}

# 8. tear down: delete all 218 Runtimes (ECR images kept)
VARIANT=terminalbench bash 6_cleanup.sh
```

- **Prereqs:** the `agentcore_p4` container, AWS creds for us-west-2, the Qwen3-4B checkpoint, 8 GPUs (4 train + 4 inference).
- **Step 5** runs the untouched machine and the reference solution on fresh sessions: keep a task only if null fails and oracle passes.
- **Step 7** reads the pool map itself; `run.sh` switches on the Terminal-Bench settings (next slide).
- **Watch** the `[demo]` lines: `open_wait_ms`, `preopen_idle_ms`, `heartbeat_session`, `solved`, `infra_error`.

Source: `demo_cpu_tasks/README.md` section 3; `5_run.sh`, `run.sh`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

The run settings size turns, tokens and the pre-open window for whole-OS tasks

Table 12 **Settings `run.sh` applies to Terminal-Bench rows (each can be overridden)**

Knob	Value	Why
`RB` × `NS`	4 × 8	32 episodes = 32 fresh microVMs per step
`HARNESS_WORKERS`	RB × NS	one worker per episode in flight
`EPISODE_MAX_TURNS` / `_TOKENS`	8 / 1024	bounded agent loop per episode
`TB_EXEC_TIMEOUT`	180 s	per bash command (builds, installs)
`MAX_TOKENS_PER_GPU`	16384	one full trajectory fits a micro-batch
`TB_PREOPEN_BATCHES`	2	upcoming batches booted ahead (slide 20)
`WARM_IDLE_TIMEOUT` / `WARM_PING_FRACTION`	900 / 0.5	heartbeat pings at 450 s idle (slide 24)

- **Fully-async rollout** (slime's only driver here): groups are drawn one at a time and the look-ahead window slides with each draw.

Source: `demo_cpu_tasks/run.sh`, README "Knobs"; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

The first session on a Runtime is warm; its GRPO siblings pay a cold boot of 6–49 s

Figure 6 Open time vs image size (4 sessions at once per Runtime)

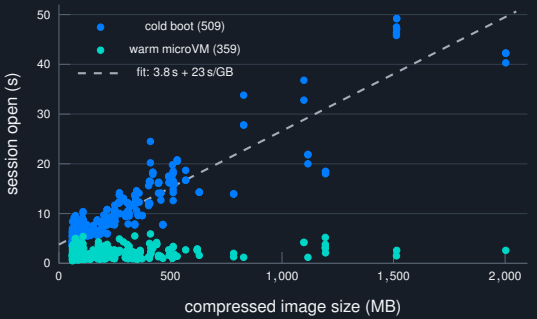


Table 13 Cold-start probe, all 218 Runtimes (2026-09-29)

Sessions	Warm	Open p50 / max
1st per Runtime	217 / 217	0.6 / 3.3 s
4 at once per Runtime	359 / 868	4.7 / 49 s
Cold opens by image size		
0–250 MB	n = 359	6.1 / 11 s
250–500 MB	107	12.1 / 25 s
500–1000 MB	26	16.7 / 34 s
1–2 GB	17	**40.3 / 49 s**

- **About one warm microVM per Runtime:** 160 of 217 served exactly 1 of the 4.
- **Cold = image pull + boot** (5.7 of 6.7 s p50), so it grows with size.
- `cron-broken-network` never booted (it breaks networking); the oracle check already drops it.

Source: `terminalbench/probe_coldstart.py` → `tasks/coldstart.csv` (1,085 of 1,090 sessions ok; conc 16), us-west-2, 2026-09-29; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

A fresh VM per episode puts a cold boot on the critical path of 7 in 8 episodes

We can't reuse a warm VM. Terminal-Bench tasks change the whole OS (apt, `/etc`, services, `/tmp`). Resetting a workdir would leak state into the next episode and corrupt the reward and the GRPO advantage. **The session is the isolation boundary**, so each episode needs a new one, and a new one can be cold.

- **Bursty:** all 8 GRPO siblings of a task start at once on the same Runtime; about 1 finds a warm VM, **7 boot cold** (slide 18).
- **Costly:** a cold open adds 6.7 s p50 to a 21 s median episode (+32 %) and up to 49 s for 1–2 GB images.
- **slime reveals little:** it only knows the episodes it has drawn; the sandbox pool can't see what comes next.

Table 14 **Options**

Option	Hides boot?	Keeps isolation?	Catch
Reuse one warm session per task + reset	✓	✗	OS state leaks → **rejected**
Standing warm pool per task	✓	✓	166 tasks × NS idle VMs
Open on pickup	✗	✓	Boot on the critical path
Look-ahead pre-open (used)	✓	✓	Guesses can be wrong; booted VMs idle

Source: `demo_cpu_tasks/terminalbench/` rollout design; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Pre-open replays slime's data order and boots the next 64 episodes' sessions

`tb_lookahead_source.py` (a `--data-source-path` plugin)

```
def get_samples(self, n):
    groups = super().get_samples(n) # slime's own draw
    rb, ns = args.rollout_batch_size, args.n_samples_per_prompt
    future = self._peek(TB_PREOPEN_BATCHES * rb) # no state moved
    drawn = [arn_for(g[0])] * ns for g in groups
    upcoming = [arn_for(s)] * ns for s in future
    pool.reserve(drawn, upcoming) # boot ahead
    return groups
```

`TerminalBenchPool` (simplified)

```
def _refill(self): # on every draw and claim
    need = Counter((demand + future)[:P]) # P = 64
    for arn in need: # open or stop per task
        while len(ready[arn]) < need[arn]:
            ready[arn].append(open_session(arn))
def _claim(self, arn): # a worker starts an episode
    s = ready[arn].popleft(); self._refill(); return s
```

1. **Look ahead without a slime change.** `_peek` replays slime's deterministic order (offset + `random.seed(seed + epoch)` shuffle) to list the next 2 batches.
2. **Bounded window.** Keep a fresh session booted for each of the next **P** = `TB_PREOPEN_BATCHES` × RB × NS = **64** episodes, pooled per task. Wrong guesses are stopped at the next draw.
3. **Claim moves the window.** Each claimed session starts booting one more.
4. **Retry on a new id** (3×); infra failures are logged as `infra_error`, never scored.

Live microVMs $\leq$ W + P = 32 + 64. Quota: **5000** active sessions, 25 new/s [Q2] (slide 36).

Source: `agentcore_slime_demo/tb_lookahead_source.py`, `terminalbench_pool.py` (`reserve`, `_refill`, `_claim`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

One batch of look-ahead is enough; each extra batch is paid as idle microVM time

Boot is hidden when $\mathbf{P \cdot E / W \geq B}$ (B = boot time, E = episode time, W = workers), and a window of at least one batch ($P \geq RB \times NS$) also covers each step's first episodes.

Table 15 **Choosing** `TB_PREOPEN_BATCHES`

`TB_PREOPEN_BATCHES`	P	Boot hidden?	Idle per pre-opened VM
0 — look-ahead off	0	No: every open is on the critical path	0
1 — one batch ahead	32	Yes, including step starts	about one step
2 — used in the smoke run	**64**	Yes, with a full batch of slack	measured 7.3 min p50, 16.5 min max

- Tune from the `[demo]` lines: `open_wait_ms` $\gg 0 \rightarrow$ raise it; `preopen_idle_ms` $\gg$ boot time $\rightarrow$ lower it.
- The idle time is also why the heartbeat is needed (slide 23).

Source: Smoke run 2026-09-23, `code/demos/_smoke_logs/lanes_tb_0923/terminalbench.demo.csv`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

With pre-open no episode waited for a boot; without it every episode did

Table 16 Warm-start probe: the same 24 episodes, batches of 6, cold vs pre-open (all rewards 1.0)

Backend	Mode	Wait p50	Episodes that waited	Idle held	Makespan
Own Runtime	without	774 ms	24 / 24	0 s	31.9 s
Own Runtime	**with**	**5 ms**	8 / 24 (first batch)	75 s	25.5 s (−20 %)
Code Interpreter	without	1715 ms	24 / 24	0 s	28.7 s
Code Interpreter	**with**	**0 ms**	6 / 24 (first batch)	70 s	24.3 s (−15 %)

Table 17 Full training smoke run with pre-open (382 episodes, P = 64)

Boot on the critical path (`open_wait_ms`)	**p50 0 · p95 0**; max 4.9 s (the first 4 episodes only)
Session open (`open_ms`)	p50 0.66 s · p95 2.5 s, 382 / 382 pre-opened · 0 retries · 0 infra errors
Price: pre-opened VM idle	p50 **7.3 min**, max 16.5 min, for a 21 s median episode

Source: Warm-start probe 2026-09-25 (`terminalbench/warmstart/`, host-only, MockPolicy); smoke run 2026-09-23 (`_smoke_logs/lanes_tb_0923`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

An idle session is silently reclaimed after 15 min, and pre-opened sessions idle that long

What AgentCore does

- A Runtime session with no invoke for `idleRuntimeSessionTimeout` (default **15 min**) is reclaimed [D3] (slide 36).
- **Silently:** the next invoke on the same session id succeeds, on a **fresh, empty sandbox**. Files, installed packages and the agent's work are gone, and nothing errors.

Why our sessions go idle

- **Pre-open:** a session is booted ahead and waits for its episode.
- **Weight sync:** while slime pushes new weights, generation stops and every open session waits.

Table 18 **Pre-open idle in the smoke run (382 sessions)**

Idle p50 / p90 / max	7.3 / 9.5 / 16.5 min
Idle past 450 s (ping point)	172
Idle past **900 s** (reclaim)	**17 (4.5 %)**

In plain English: without a keep-alive, 1 in 22 pre-opened machines would be wiped before its episode, and the episode would score 0 on an empty machine without anyone noticing.

Source: AgentCore docs [D3]; smoke run 2026-09-23, `terminalbench.demo.csv` (`preopen_idle_ms`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

One process-wide keeper pings any session idle past half the timeout

`session_keeper.py` (simplified)

```
class SessionKeeper: # one per rollout process
    def register(self, sid, pinger, idle_timeout): ...
    def touch(self, sid): # every real invoke
        self._sessions[sid].last = now()
    async def _run(self): # background loop
        while True:
            for s in self._sessions.values():
                if now() - s.last >= s.idle_timeout * 0.5:
                    try:
                        await s.pinger() # exec("true")
                        s.last = now()
                    except Exception: # retry next tick,
                        s.ping_failures += 1 # clock NOT reset
            await asyncio.sleep(tick)
```

`TerminalBenchPool._open_session`

```
async def _ping(s=sb): await s.exec("true", timeout=30)
keeper.register(sb.session_id, _ping, WARM_IDLE_TIMEOUT)
```

- **Outside the harness.** No agent loop grows its own keep-alive; every session is registered when it opens and unregistered when it stops.
- **Cheap ping:** `exec "true"`, one invoke that resets AgentCore's idle clock.
- **Zero pings while busy:** real episode traffic refreshes the same clock, so only idle sessions are pinged.
- **A failed ping doesn't reset the clock**, so it retries at the next tick instead of a full interval later.
- **Always on** for Terminal-Bench rows; `HEARTBEAT=1` (default) for the rest. New Runtimes also get `RUNTIME_IDLE_TIMEOUT=3600` as a backstop.

Source: `agentcore_slime_demo/session_keeper.py` (`SessionKeeper._run`), `terminalbench_pool.py` (`_open_session`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

With the heartbeat, sessions survived the idle gap and 0 of 382 were lost

Probe: Runtime `acs_warmstart_cpu` with a **60 s** idle timeout (so the test fits in minutes); sessions then sit idle 150 s.

Table 19 **Warm-start probe: heartbeat vs idle reclaim**

Backend	Heartbeat	Sessions surviving	Resume
Own Runtime	on (14 pings)	**3 / 3**	104 ms, files intact
Own Runtime	off	0 / 3	**Silently reclaimed:** same id, fresh empty sandbox
Code Interpreter	on / off	0 / 3	Ends at its `sessionTimeoutSeconds` regardless of pings

Table 20 **Full training smoke run (idle timeout 900 s, pings at 450 s)**

Heartbeat pings	**348**, on 214 sessions · 0 failed
Sessions idle past the 900 s timeout	17, all **alive** at claim · 0 lost episodes, 0 infra errors

Source: Warm-start probe 2026-09-25 (`terminalbench/warmstart/`); smoke run 2026-09-23 (`terminalbench.demo.csv`: `heartbeat_session`, `total_pings`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

The smoke run answers three questions: yes, yes, and not yet

Table 21 **Smoke run 2026-09-23: Qwen3-4B, RB = 4 × NS = 8, 10 steps + 1 eval on 31 held-out tasks, ≈ 70 min**

Question	Answer	Evidence	Slide
1. Does one-image-per-task RL run end to end on AgentCore?	**Yes**	382 episodes (train + eval) on 382 fresh microVMs, 71 tasks; **0** infra errors, **0** retries, **0** grading timeouts	slide 27
2. Is the sandbox off the critical path?	**Yes**	boot wait p95 **0 ms** (pre-open); **0** of 17 over-idle sessions lost (heartbeat)	slides 22–25
3. Does the policy learn?	**Not yet**	4 / 382 solved; 8 of 10 steps had a zero gradient; reward too sparse for GRPO	slides 28–29

In plain English: the plumbing works and costs no waiting time; the task set is too hard for a 4B model with all-or-nothing reward, so the next run is about the reward, not the sandbox.

Source: `code/demos/_smoke_logs/lanes_tb_0923/terminalbench.{csv,demo.csv,eval.csv}`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every one of 382 episodes ran and was graded; the trainer waited on rollout

Table 22 System health (all episodes)

Episodes / distinct tasks	382 / 71
Fresh session per episode	382 / 382
Open attempts > 1 · infra errors	0 · **0**
Grading timeouts	0
Heartbeat pings / failed	348 / 0

Table 23 Where an episode's time goes (p50)

Wait for a session (boot)	0 s
Agent loop in the microVM	9.0 s
Grading (inject tests, pytest)	9.7 s
Whole episode (p90)	20.9 s (63.7 s)

Table 24 Step time

Steps 1–8 p50	203 s
Steps 9–10 (with the eval pass)	640 / 1,229 s
Share of step time the trainer waits	77–97 %

- **Rollout-bound, as expected:** a step waits for the slowest of its 32 episodes (p95 122 s, max 8.3 min), not for boots.
- **Grading is half the episode:** every grade installs `uv` + pytest from the internet (NAT egress); baking them into the image would cut it.

Source: `terminalbench.demo.csv` (`open_wait_ms`, `agent_ms`, `grade_ms`, `episode_ms`, `infra_error`), `terminalbench.csv` (`perf/step_time`, `perf/wait_time_ratio`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Partial progress is common (42 % pass a test) but full solves are rare (4 of 382)

Figure 7 Per training step: share of episodes that pass every hidden test vs at least one



- **Solves:** `acl-permissions-inheritance` 3 of 8 at step 3, `git-leak-recovery` 1 of 8 at step 8. Held-out eval (31 tasks): **0** solved.
- **No trend over 10 steps**, in either measure. Partial progress is flat across difficulty (easy 41 %, medium 42 %, hard 43 %).

Source: `terminalbench.demo.csv` (`solved`, `tests_passed`, per step; step 10 includes the rows drawn alongside eval), `terminalbench.eval.csv`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

All-or-nothing reward left 76 of 78 GRPO groups with no gradient; test-level credit would give 28

Figure 8 Fraction of hidden tests passed, 382 episodes

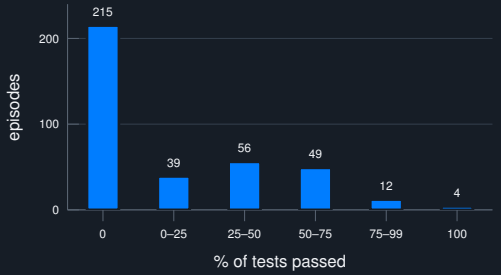


Table 25 Why episodes stopped

Hit the 8-turn budget	170 (45 %)
Hit the 1024-token turn limit	112 (29 %)
Agent said it was done	100 (26 %)

Table 26 GRPO groups (one task × 8 samples in a step)

Groups	78
Mixed under binary reward (some 1, some 0)	**2**
Mixed under test-fraction reward	**28**
Steps with non-zero grad norm	2 of 10

- **GRPO learns only from mixed groups:** an all-0 group has zero advantage, so 8 of 10 steps updated nothing.
- **The budget binds:** 74 % of episodes ran out of turns or tokens before finishing.
- **Fix the signal first:** partial credit from `tests_passed` would turn 2 useful groups into 28.

Source: `terminalbench.demo.csv` (`tests_passed/tests_total`, `stop`, grouped by step and task), `terminalbench.csv` (`train/grad_norm`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Next: fix the reward and the budget, then halve the idle cost

Table 27 Next steps, in order

	Change	Why (evidence)
1	Partial credit from `tests_passed` / `tests_total`	2 → 28 of 78 groups with a gradient (slide 29)
2	Larger turn and token budget (e.g. 16 turns, 2048 tokens)	74 % of episodes stopped on a budget
3	Easy-first curriculum over the 166 tasks	4B model solves 1 %; start where it sometimes succeeds
4	`TB_PREOPEN_BATCHES` = 1	microVMs idle ≈ 95 % of their life; one batch still hides the boot (slide 21)
5	Bake `uv` + pytest into each image	grading is 9.7 s of a 20.9 s episode, mostly the install

The next deploy rebuilds every image once (≈ 1 h), because the server layer changed in the merge into `demo_cpu_tasks`.

Source: Smoke run 2026-09-23, `code/demos/_smoke_logs/lanes_tb_0923`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

A SWE-bench row is a repo at one commit with its environment; Verified has 500 rows

Table 28 The task

Input	A real GitHub issue + the repo checked out at `base_commit`, dependencies installed in conda env `testbed`
Agent's job	Explore, edit, run code: multi-turn `bash` / `read_file` / `write_file`
Reward	Hidden `test_patch` applied after the agent stops; solved iff every FAIL_TO_PASS and PASS_TO_PASS test passes
Image	One prebuilt image per row, `swebench/sweb.eval.x86_64.<id>`

Table 29 Dataset size

Verified (used)	**500** rows, 12 Python repos (django 231, sympy 75, . . .)
Full test / SWE-Gym / SWE-smith	2,294 / 2,438 / 59,136 rows
Our demo today	20 rows (16 train, 4 eval)

- **Same pattern as Terminal-Bench:** one row = one task + one image = one Runtime; fresh session per episode.
- **Different shape:** the environment is a Python repo + conda env, not a whole custom OS, and the images are about 8× larger.
- **Tests run inside the row's env**, so a row's image must be the one SWE-bench validated.

Source: `princeton-nlp/SWE-bench_Verified`, `SWE-Gym/SWE-Gym`, `SWE-bench/SWE-smith` (Hugging Face); `demo_cpu_swebench/data/`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

500 rows need 500 distinct images, which collapse to 80 environments

Table 30 Rows, images and environments

	Terminal-Bench	SWE-bench Verified	Note
Rows	229	**500**	one per task / instance
Images (one per row)	229	**500**	official, one per instance id
Distinct images	203 of 218 built	**500**	every SWE-bench image bakes its own commit
Distinct environments	—	**80**	(repo, version) pairs; django alone is 9
Size p50 / max (compressed)	127 MB / 2.0 GB	1.08 GB / 2.95 GB	SWE-bench: official amd64
Total size	48 GB	619 GB	$\approx 3\times$ that on disk

- **Why one image per row:** each row pins a different commit, and the dependency set changes with the repo version; the tests are only valid in that environment.
- **80 environments** make one image per environment possible (checkout the commit at episode start): 500 $\rightarrow$ 80 Runtimes, the way to scale to SWE-Gym under the 1000-Runtime quota.

Source: `demo_cpu_swebench/swebench/verified_image_sizes.csv` (`survey_image_sizes.py`, Docker Hub + GHCR); Terminal-Bench `images.csv`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Every prebuilt arm64 image fits the 2 GB cap; only matplotlib's amd64 images are over it

Figure 9 Compressed image size per Verified row (100 MB bins): arm64 top, amd64 bottom



arm64 is the architecture that matters for microVMs; amd64 is what the m5 Instance lane runs and an upper bound for arm64 rebuilds. arm64 is about 0.83× amd64 at the median.

Source: `demo_cpu_swebench/swebench/verified_image_sizes.csv`: Docker Hub `swebench/sweb.eval.{arm64,x86_64}.*`, GHCR `epoch-research/swe-bench.eval.arm64.*`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

420 of 500 rows can run on AgentCore microVMs today; 27 never can

Table 31 Where each Verified row can run

Status	Rows	Why
✔ microVM now	420	prebuilt arm64 image exists, $\leq$ 1.77 GB + 54 MB wrapper
🔧 microVM after rebuild	53	no arm64 image; amd64 $\leq$ 2 GB, so an arm64 build likely fits
✖ not on a microVM	27	no arm64 image and amd64 $>$ 2 GB (matplotlib 3.4–3.7)

Table 32 The 80 rows without arm64, by repo

matplotlib 27 · scikit-learn 25 · xarray 22 · astropy 4 · django 2

The limits that decide it

- **arm64 only** on microVMs [D1].
- **2048 MiB compressed image cap**, not adjustable; probed on AWS: 2,042 MiB passes, 2,062 MiB is rejected at `CreateAgentRuntime`.
- **1000 Runtimes per account:** fine for 500 rows, not for SWE-Gym's 2,438 → one image per environment.

The 27 still run on AgentCore

- On an **m5 capacity-provider Instance**: amd64, no size cap, but one whole instance per session and a ≈ 90 s boot.

Source: `verified_image_sizes.csv`; cap probe `probe_image_cap.py` (`swebench/capprobe*.csv`, 2026-09-28); quotas [Q1][Q2] (slide 36); `demo_cpu_swebench/README.md`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Appendix

References

AgentCore quotas and docs back every limit used in this deck

Table 33 References

Ref	Source	Supports
[Q1]	AgentCore quotas: docs.aws.amazon.com/bedrock-agentcore/latest/devguide/bedrock-agentcore-limits.html	1000 Runtimes/account · image $\leq$ 2 GB · 15-min request timeout · 15-min idle timeout
[Q2]	`aws service-quotas list-service-quotas --service-code bedrock-agentcore` (us-west-2, 2026-09-23)	Active Session Workloads = 5000 (L-3E5722B2) · new sessions 25/s (L-8EE2AEA2) · image 2048 MB (L-0A9E32B3)
[D1]	AgentCore Runtime developer guide	microVMs are arm64 only
[D3]	docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html	Session = dedicated microVM; 15-min idle, 8 h max lifetime

Source: AWS documentation and service-quotas CLI, us-west-2, 2026-09-23; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Thank you

Kaige Yang · Senior Applied Scientist, AWS